

Natural Language Processing

05. Modèles Linéaires

Kenza BOUZIDI

`kenza.bouzidi.ext@u-pec.fr`

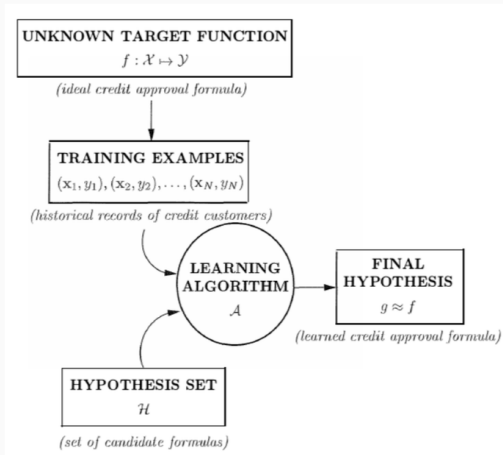
09 Janvier 2026

1. Apprentissage supervisé
2. Modèles linéaires
3. Représentations
4. Apprentissage

Apprentissage supervisé

- L'essence de l'apprentissage automatique supervisé (supervised machine learning) est la création de mécanismes capables d'observer des exemples et de produire des généralisations. [Goldberg, 2017]
- Nous concevons un algorithme dont l'entrée est un ensemble d'exemples étiquetés , et dont la sortie est une fonction (ou un programme) qui reçoit une instance et produit l'étiquette ou label désirée.
- Exemple : si la tâche consiste à distinguer les e-mails *spam* des e-mails *non-spam*, les exemples étiquetés sont des e-mails annotés comme *spam* et des e-mails annotés comme *non-spam*.
- On s'attend à ce que la fonction résultante produise des prédictions correctes des étiquettes, y compris pour des instances qu'elle n'a jamais vues durant l'entraînement.
- Cette approche est différente de la conception manuelle d'un algorithme pour réaliser la tâche (par exemple, les systèmes à règles écrites à la main).

Apprentissage supervisé

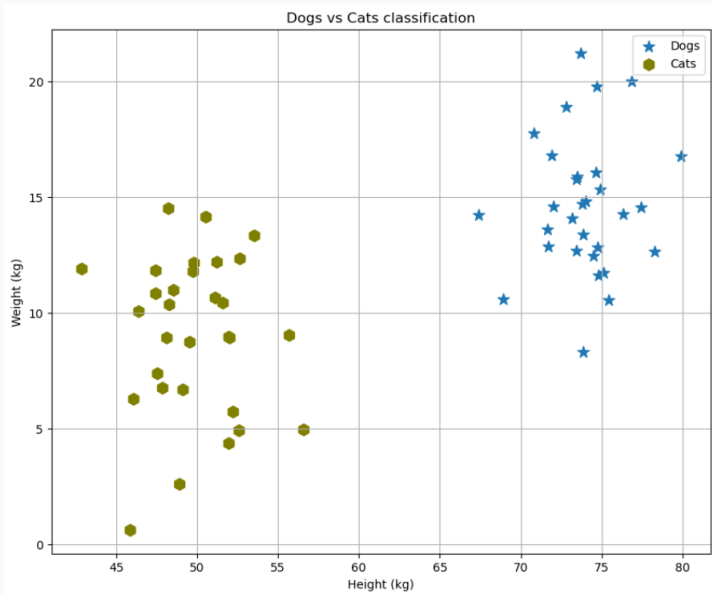


¹Source : [Abu-Mostafa et al., 2012]

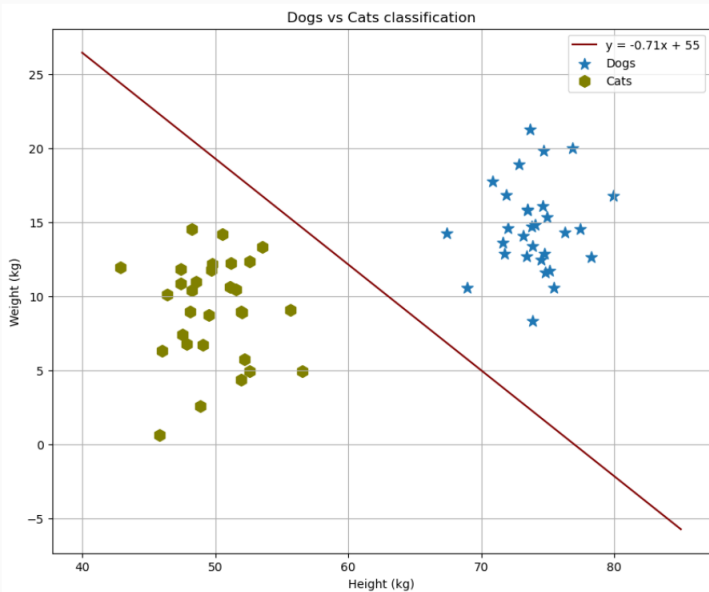
Fonctions paramétrées

- Rechercher parmi l'ensemble de toutes les fonctions possibles est un problème très difficile (et plutôt mal défini). [Goldberg, 2017]
- On se restreint donc souvent à la recherche au sein de familles spécifiques de fonctions.
- Exemple : l'espace de toutes les fonctions linéaires ayant d_{in} entrées et d_{out} sorties.
- Ces familles de fonctions sont appelées des **classes d'hypothèses**.
- En nous restreignant à une classe d'hypothèses donnée, nous injectons dans l'apprentissage un **biais inductif**.
- Biais inductif : ensemble d'hypothèses ou d'assumptions sur la forme de la solution recherchée.
- Certaines classes d'hypothèses permettent des procédures efficaces pour rechercher la solution. [Goldberg, 2017]

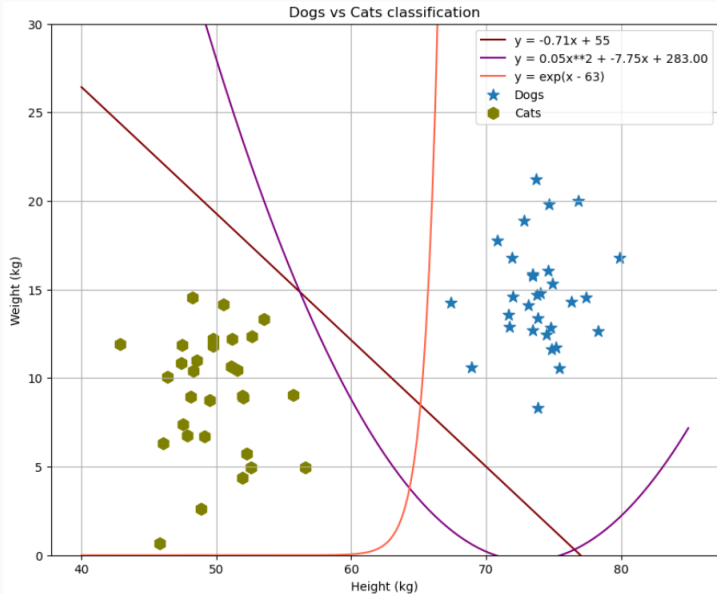
Problème de classification



Problème de classification : modèle linéaire



Problème de classification : autres alternatives



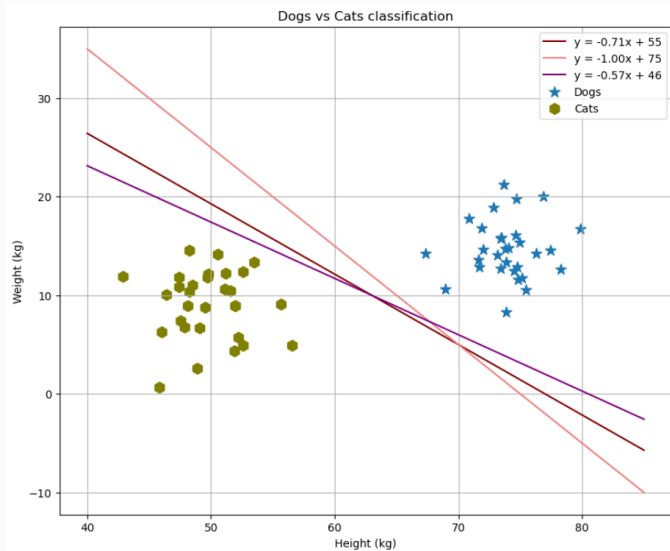
Modèles linéaires

- Une classe d'hypothèses très courante est celle des fonctions linéaires de grande dimension :

$$f(\vec{x}) = \vec{x} \cdot W + \vec{b}$$
$$\vec{x} \in \mathcal{R}^{d_{in}} \quad W \in \mathcal{R}^{d_{in} \times d_{out}} \quad \vec{b} \in \mathcal{R}^{d_{out}} \quad (1)$$

- Le vecteur $\vec{x}$ correspond à l'entrée (*input*) de la fonction.
- La matrice W et le vecteur $\vec{b}$ sont les **paramètres** du modèle.
- L'objectif de learner est de fixer les valeurs des paramètres W et $\vec{b}$ de telle sorte que la fonction se comporte comme attendu sur un ensemble d'entrées $\vec{x}_{1:k} = \vec{x}_1, \dots, \vec{x}_k$ et les sorties désirées correspondantes $\vec{y}_{1:k} = \vec{y}_1, \dots, \vec{y}_k$.
- Ainsi, la recherche dans l'espace des fonctions est ramenée à une recherche dans l'espace des paramètres. [Goldberg, 2017]

Problème de classification : modèle linéaire (plusieurs alternatives)



Exemple : Détection de langue

- Considérons la tâche de distinguer des documents écrits en anglais de documents écrits en allemand.
- Il s'agit d'un problème de classification binaire :

$$f(\vec{x}) = \vec{x} \cdot \vec{w} + b \quad (2)$$

où $d_{out} = 1$, $\vec{w}$ est un vecteur et b est un scalaire.

- La fonction linéaire peut prendre des valeurs dans $[-\infty, \infty]$.
- Pour l'utiliser en classification binaire, il est courant de passer la sortie $f(x)$ à travers la fonction *sign*, qui mappe les valeurs négatives vers -1 (classe négative) et les valeurs non négatives vers +1 (classe positive).

Exemple : Détection de langue

- Les fréquences de lettres constituent de bons prédicteurs (*features*) pour cette tâche.
- Encore plus informatifs sont les comptages de bigrams de lettres, c'est-à-dire des paires de lettres consécutives.
- On pourrait penser que les mots seraient également de bons prédicteurs, en utilisant une représentation Bag of Words des documents.
- Les lettres ou les bigrams de lettres sont beaucoup plus robustes.
- Il est probable qu'un nouveau document contienne des mots jamais observés dans l'ensemble d'entraînement.
- En revanche, un document sans aucun des bigrams distinctifs est beaucoup moins probable. [Goldberg, 2017]

Exemple : Détection de langue

- On suppose que nous avons un alphabet de 28 lettres (a–z, espace, et un symbole spécial pour tous les autres caractères, y compris chiffres, ponctuation, etc.).
- Les documents sont représentés comme des vecteurs de dimension 28×28 , soit $\vec{x} \in \mathcal{R}^{784}$.
- Chaque entrée $\vec{x}_{[i]}$ représente le comptage d'une combinaison de lettres particulière dans le document, normalisé par la longueur du document.
- Par exemple, en notant $\vec{x}_{ab}$ l'entrée de $\vec{x}$ correspondant au bigramme de lettres ab :

$$\vec{x}_{ab} = \frac{\#ab}{|D|} \quad (3)$$

où $\#ab$ est le nombre de fois que le bigram ab apparaît dans le document, et $|D|$ est le nombre total de bigrams dans le document (longueur du document - 1).

Exemple : Détection de langue

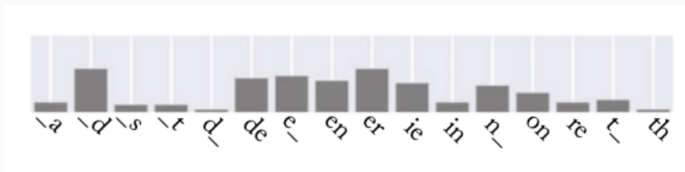


Histogrammes de bigrams de caractères pour des documents en anglais (à gauche, bleu) et en allemand (à droite, vert). Les underscores représentent les espaces. Seuls les bigrams de caractères les plus fréquents sont affichés.

²Source : [Goldberg, 2017]

Exemple : Détection de langue

- La figure précédente montrait des motifs clairs dans les données, et, pour un nouvel élément, comme :



- On pourrait probablement dire qu'il est plus similaire au groupe allemand qu'au groupe anglais (observez la fréquence de "th" et "ie").
- On ne peut pas utiliser une règle définitive unique telle que si le document contient "th", il est anglais ou si le document contient "ie", il est allemand .
- Bien que les textes allemands contiennent beaucoup moins de "th" que l'anglais, le "th" peut et apparaît dans les textes allemands, et de même, la combinaison "ie" peut apparaître dans les textes anglais.

Exemple : Détection de langue

- La décision nécessite de pondérer les différents facteurs les uns par rapport aux autres.
- On peut formaliser le problème dans un cadre d'apprentissage automatique en utilisant un modèle linéaire :

$$\begin{aligned}\hat{y} &= \text{sign}(f(\vec{x})) = \text{sign}(\vec{x} \cdot \vec{w} + b) \\ &= \text{sign}(\vec{x}_{aa} \times \vec{w}_{aa} + \vec{x}_{ab} \times \vec{w}_{ab} + \vec{x}_{ac} \times \vec{w}_{ac} \cdots + b)\end{aligned}\tag{4}$$

- Un document sera considéré comme anglais si $f(\vec{x}) \geq 0$ et comme allemand sinon.

Intuition

1. L'apprentissage doit assigner de grandes valeurs positives aux entrées $\vec{w}$ correspondant aux paires de lettres beaucoup plus fréquentes en anglais qu'en allemand (ex. "th").
2. Il doit assigner des valeurs négatives aux paires de lettres beaucoup plus fréquentes en allemand qu'en anglais (ex. "ie", "en").
3. Enfin, il doit attribuer des valeurs proches de zéro aux paires de lettres qui sont soit communes, soit rares dans les deux langues.

Classification binaire log-linéaire

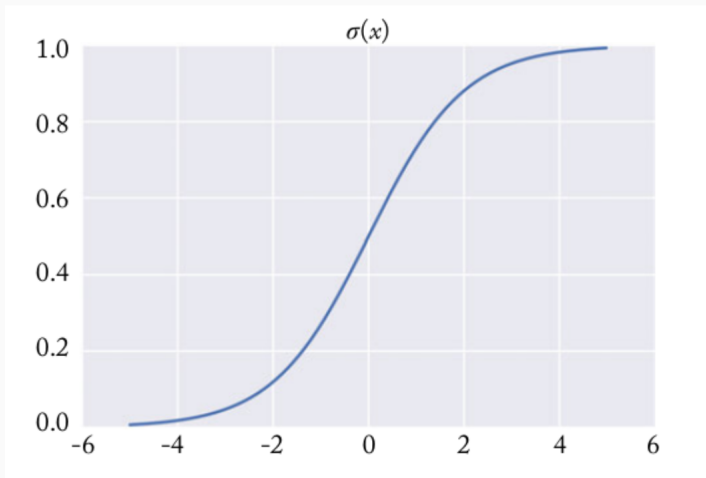
- La sortie $f(\vec{x})$ est dans l'intervalle $[-\infty, \infty]$, et nous la convertissons en l'une des deux classes $\{-1, +1\}$ en utilisant la fonction *sign*.
- Cela convient si tout ce qui nous intéresse est la classe attribuée.
- On peut également s'intéresser à la **confiance** de la décision, c'est-à-dire à la probabilité que le classifieur attribue à chaque classe.
- Une alternative qui permet de le faire consiste à mapper la sortie dans l'intervalle $[0, 1]$, en passant la sortie à travers une **fonction d'activation sigmoïde** $\sigma(x)$:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (5)$$

ce qui donne :

$$\hat{y} = \sigma(f(\vec{x})) = \frac{1}{1 + e^{-\vec{x} \cdot \vec{w} - b}} \quad (6)$$

La fonction sigmoïde



La fonction sigmoïde

- La fonction sigmoïde est ****monotone croissante**** et mappe les valeurs dans l'intervalle $[0, 1]$, avec 0 mappé à $\frac{1}{2}$.
- Lorsqu'elle est utilisée avec une fonction de perte appropriée (discutée plus tard), les prédictions binaires faites via le modèle log-linéaire peuvent être interprétées comme des ****estimations de probabilité d'appartenance à la classe**** :

$$\sigma(f(\vec{x})) = P(\hat{y} = 1|\vec{x}) \quad \text{probabilité que } \vec{x} \text{ appartienne à la classe positive.} \quad (7)$$

- On obtient également $P(\hat{y} = 0|\vec{x}) = 1 - P(\hat{y} = 1|\vec{x}) = 1 - \sigma(f(\vec{x}))$
- Plus la valeur est proche de 0 ou 1, plus le modèle est **certain** de sa prédiction de classe, une valeur de 0.5 indiquant **incertitude**.
- En général, on fixe un seuil $T = 0.5$. Si $P(\hat{y}|\vec{x}) > T$, alors on dit que $\hat{y} = 1$, sinon 0. **Très important** : $T = 0.5$ est le seuil par défaut, mais il peut (et doit !) être ajusté selon le problème. Parfois, un faux négatif est plus coûteux qu'un faux positif ou vice-versa.

Classification multi-classes

- La plupart des problèmes de classification sont de nature multi-classes : les exemples sont assignés à l'une des k classes différentes.
- Exemple : on nous donne un document et on doit le classer dans l'une des six langues possibles : anglais, français, allemand, italien, espagnol, autre.
- Solution possible : considérer six vecteurs de poids $\vec{w}_{EN}$, $\vec{w}_{FR}$, ... et des biais (un pour chaque langue).
- On prédit la langue correspondant au score le plus élevé :

$$\hat{y} = f(\vec{x}) = \operatorname{argmax}_{L \in \{EN, FR, GR, IT, SP, O\}} \vec{x} \cdot \vec{w}_L + b_L \quad (8)$$

- Les six ensembles de paramètres $\vec{w}_L \in \mathcal{R}^{784}$ et b_L peuvent être organisés en une matrice $W \in \mathcal{R}^{784 \times 6}$ et un vecteur $\vec{b} \in \mathcal{R}^6$, et l'équation peut être réécrite comme suit :

$$\begin{aligned}\vec{y} &= f(\vec{x}) = \vec{x} \cdot W + \vec{b} \\ \text{prédiction} = \hat{y} &= \operatorname{argmax}_i \vec{y}_{[i]}\end{aligned}\tag{9}$$

- Ici, $\vec{y} \in \mathcal{R}^6$ est un vecteur des scores attribués par le modèle à chaque langue, et on détermine la langue prédite en prenant l'argmax des entrées de $\vec{y}$ (la colonne ayant la valeur la plus élevée).

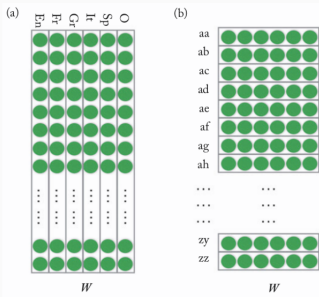
- Considérons le vecteur $\vec{\hat{y}}$ obtenu en appliquant un modèle entraîné à un document.
- Ce vecteur peut être considéré comme une représentation du document.
- Il capture les propriétés du document qui nous intéressent : les scores attribués aux différentes langues.
- La représentation $\vec{\hat{y}}$ contient strictement plus d'informations que la simple prédiction $\operatorname{argmax}_i \hat{y}_{[i]}$.
- Exemple : $\vec{\hat{y}}$ peut permettre de distinguer des documents dont la langue principale est l'allemand, mais qui contiennent également une quantité non négligeable de mots français.
- Le clustering de documents basé sur $\vec{\hat{y}}$ pourrait aider à découvrir des documents écrits dans des dialectes régionaux ou par des auteurs multilingues.

Représentations

- Les vecteurs $\vec{x}$ contenant les comptages normalisés de bigrams de lettres pour les documents sont également des représentations des documents.
- Ils contiennent probablement un type d'information similaire à celui des vecteurs $\vec{y}$.
- Cependant, les représentations $\vec{y}$ sont plus compactes (6 entrées au lieu de 784) et plus spécialisées pour l'objectif de prédiction de la langue.
- Un clustering basé sur les vecteurs $\vec{x}$ révélerait probablement des similitudes entre documents qui ne sont pas dues à un mélange particulier de langues, mais plutôt au sujet du document ou au style d'écriture.

Représentations

- La matrice entraînée $W \in \mathcal{R}^{784 \times 6}$ peut également être considérée comme contenant des représentations apprises.
- On peut considérer deux vues de W : par colonnes ou par lignes.



Deux vues de la matrice W : (a) chaque colonne correspond à une langue ; (b) chaque ligne correspond à un bigramme de lettres. Source : [Goldberg, 2017]..

- Une colonne de W peut être considérée comme un vecteur 784 dimensions représentant une langue en termes de ses motifs caractéristiques de bigrams de lettres.
- On peut ensuite regrouper (cluster) les 6 vecteurs de langues selon leur similarité.
- Chacune des 784 lignes de W fournit un vecteur 6 dimensions représentant ce bigramme en termes des langues qu'il influence.

- Les représentations sont au cœur du deep learning.
- On peut dire que la principale puissance du deep learning réside dans sa capacité à apprendre de bonnes représentations.
- Dans le cas linéaire, ces représentations sont interprétables.
- On peut donner une interprétation significative à chaque dimension du vecteur de représentation.
- Par exemple : chaque dimension correspond à une langue particulière ou à un bigramme de lettres.

- Les modèles de deep learning, en revanche, apprennent souvent une cascade de représentations de l'entrée qui se construisent les unes sur les autres.
- Ces représentations ne sont souvent pas interprétables.
- On ne sait pas quelles propriétés de l'entrée elles capturent.
- Cependant, elles restent très utiles pour effectuer des prédictions.

Représentation en vecteurs one-hot

- Le vecteur d'entrée $\vec{x}$ dans notre exemple de classification de langue contient les comptages normalisés de bigrams dans le document D .
- Ce vecteur peut être décomposé en une moyenne de $|D|$ vecteurs, chacun correspondant à une position particulière i dans le document :

$$\vec{x} = \frac{1}{|D|} \sum_{i=1}^{|D|} \vec{x}^{D[i]} \quad (10)$$

- Ici, $D[i]$ est le bigramme à la position i du document.
- Chaque vecteur $\vec{x}^{D[i]} \in \mathcal{R}^{784}$ est un vecteur one-hot.

Représentation en vecteurs one-hot

- Un vecteur one-hot : toutes les entrées sont nulles sauf celle correspondant au bigramme $D_{[i]}$, qui vaut 1.
- Le vecteur résultant $\vec{x}$ est couramment appelé un **bag of bigrams moyen** (ou plus généralement un bag of words moyen, ou simplement bag of words).
- Les représentations bag-of-words (BOW) contiennent des informations sur l'identité de tous les "mots" (ici, les bigrams) du document, sans tenir compte de leur ordre.
- Une représentation one-hot peut être considérée comme un **bag of a single word**.

Représentation en vecteurs one-hot

Diagram illustrating the one-hot representation of words. The words are mapped to binary vectors where the position of the '1' corresponds to the word's index in the vocabulary.

Rome = $[1, 0, 0, 0, 0, 0, \dots, 0]$

Paris = $[0, 1, 0, 0, 0, 0, \dots, 0]$

Italy = $[0, 0, 1, 0, 0, 0, \dots, 0]$

France = $[0, 0, 0, 1, 0, 0, \dots, 0]$

Vecteurs one-hot de mots. Source :

<https://medium.com/@athif.shaffy/one-hot-encoding-of-text-b69124bef0a7>.

Représentation en vecteurs one-hot

- À partir de l'équation 9, on sait que $\vec{y} = f(\vec{x}) = \vec{x} \cdot W + \vec{b}$.
- Oublions un instant les biais $\vec{b}$ et concentrons-nous uniquement sur le terme $\vec{x} \cdot W$.
- Puisque $\vec{x}$ peut être décomposé comme une somme de vecteurs one-hot :

$$\begin{aligned}\vec{y} &= \vec{x} \cdot W \\ &= \left(\frac{1}{|D|} \sum_{i=1}^{|D|} \vec{x}^{D[i]} \right) \cdot W \\ &= \frac{1}{|D|} \left(\sum_{i=1}^{|D|} \vec{x}^{D[i]} \cdot W \right) \\ &= \frac{1}{|D|} \sum_{i=1}^{|D|} W^{D[i]}\end{aligned}\tag{11}$$

- Autrement dit, la représentation du document peut être comprise comme une somme de représentations de mots. On parle alors de continuous bag of words (CBOW), un concept que nous reverrons dans quelques séances.

Classification multi-classes log-linéaire

- Dans le cas binaire, nous avons transformé la prédiction linéaire en une estimation de probabilité en la faisant passer par la fonction sigmoïde, ce qui donne un modèle log-linéaire.
- L'analogue dans le cas multi-classes consiste à faire passer le vecteur de scores par la fonction softmax :

$$\text{softmax}(\vec{x})_{[i]} = \frac{e^{\vec{x}_{[i]}}}{\sum_j e^{\vec{x}_{[j]}}} \quad (12)$$

Ce qui donne :

$$\begin{aligned} \vec{\hat{y}} &= \text{softmax}(\vec{x} \cdot W + \vec{b}) \\ \hat{y}_{[i]} &= \frac{e^{(\vec{x} \cdot W + \vec{b})_{[i]}}}{\sum_j e^{(\vec{x} \cdot W + \vec{b})_{[j]}}} \end{aligned} \quad (13)$$

- La transformation softmax force les valeurs de $\vec{\hat{y}}$ à être positives et à sommer à 1, ce qui permet de les interpréter comme une distribution de probabilités.

Apprentissage

- Lors de l'apprentissage d'une fonction paramétrée (par exemple un modèle linéaire ou un réseau de neurones), on définit une fonction de perte $L(\hat{y}, y)$, qui mesure la perte associée à la prédiction $\hat{y}$ lorsque la sortie réelle est y .

$$L(f(\vec{x}; \Theta), y)$$

- On utilise le symbole Θ pour désigner l'ensemble des paramètres du modèle (par exemple $W, \vec{b}$).
- L'objectif de l'apprentissage est alors de minimiser la perte sur l'ensemble des exemples d'entraînement.
- Formellement, une fonction de perte $L(\hat{y}, y)$ associe une valeur numérique (un scalaire) à une prédiction $\hat{y}$ étant donné la sortie attendue y .

- La fonction de perte doit atteindre sa valeur minimale lorsque la prédiction est correcte.
- On peut également définir une perte globale sur l'ensemble du corpus, par rapport aux paramètres Θ , comme la perte moyenne sur tous les exemples d'entraînement :

$$\mathcal{L}(\Theta) = \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i)$$

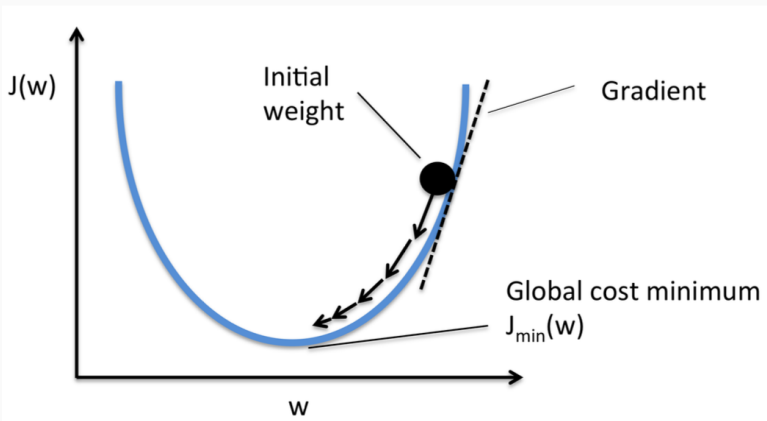
- L'objectif de l'algorithme d'apprentissage est alors de fixer les valeurs des paramètres Θ de manière à minimiser la valeur de $\mathcal{L}$.

$$\hat{\Theta} = \operatorname{argmin}_{\Theta} \mathcal{L}(\Theta) = \operatorname{argmin}_{\Theta} \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i)$$

Optimisation par gradient

- Les fonctions sont entraînées à l'aide de méthodes basées sur le gradient.
- Ces méthodes fonctionnent en calculant de manière répétée une estimation de la perte L sur l'ensemble d'entraînement.
- La méthode d'apprentissage calcule les gradients des paramètres Θ par rapport à l'estimation de la perte, puis déplace les paramètres dans la direction opposée au gradient.
- Les différentes méthodes d'optimisation se distinguent par la manière dont l'erreur est estimée et par la façon de définir le déplacement dans la direction opposée au gradient.
- Si la fonction est convexe, l'optimum trouvé sera global.
- Sinon, le processus garantit seulement la convergence vers un optimum local.

Descente de gradient



⁰Source : <https://sebastianraschka.com/images/faq/closed-form-vs-gd/ball.png>

Descente de gradient

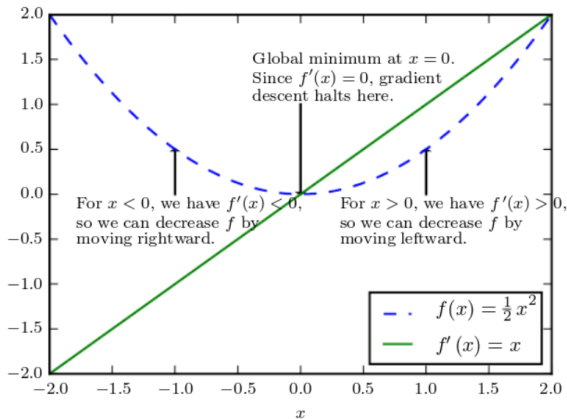


Figure 4.1: Gradient descent. An illustration of how the gradient descent algorithm uses the derivatives of a function to follow the function downhill to a minimum.

Descente de gradient stochastique en ligne

- Tous les paramètres sont initialisés avec des valeurs aléatoires (Θ).
- Pour chaque exemple d'entraînement (x, y) , on calcule la perte L avec les valeurs courantes de Θ .
- On met ensuite à jour les paramètres selon la règle suivante jusqu'à convergence :
- $\Theta_i \leftarrow \Theta_i - \eta \frac{\partial L}{\partial \Theta_i}(\hat{y}, y)$ (pour tous les paramètres Θ_i)

Algorithm 2.1 Online stochastic gradient descent training.

Input:

- Function $f(x; \Theta)$ parameterized with parameters Θ .
- Training set of inputs $x_1, \dots, x_n$ and desired outputs $y_1, \dots, y_n$.
- Loss function L .

```
1: while stopping criteria not met do
2:   Sample a training example  $x_i, y_i$ 
3:   Compute the loss  $L(f(x_i; \Theta), y_i)$ 
4:    $\hat{g} \leftarrow$  gradients of  $L(f(x_i; \Theta), y_i)$  w.r.t  $\Theta$ 
5:    $\Theta \leftarrow \Theta - \eta_t \hat{g}$ 
6: return  $\Theta$ 
```

⁰Source : [Goldberg, 2017]

Descente de gradient stochastique en ligne

- Le taux d'apprentissage peut soit rester fixe pendant tout le processus d'entraînement, soit décroître en fonction du pas de temps t .
- L'erreur calculée à l'étape 3 est basée sur un seul exemple d'entraînement ; il s'agit donc seulement d'une estimation grossière de la perte globale du corpus L que l'on cherche à minimiser.
- Le bruit présent dans le calcul de la perte peut conduire à des gradients imprécis (un seul exemple peut fournir une information bruitée).

Descente de gradient stochastique par mini-lots

- Une manière courante de réduire ce bruit consiste à estimer l'erreur et les gradients à partir d'un échantillon de m exemples.
- Cela conduit à l'algorithme de descente de gradient stochastique par mini-lots (mini-batch SGD).

Algorithm 2.2 Minibatch stochastic gradient descent training.

Input:

- Function $f(x; \Theta)$ parameterized with parameters Θ .
- Training set of inputs $x_1, \dots, x_n$ and desired outputs $y_1, \dots, y_n$.
- Loss function L .

```
1: while stopping criteria not met do
2:   Sample a minibatch of  $m$  examples  $\{(x_1, y_1), \dots, (x_m, y_m)\}$ 
3:    $\hat{g} \leftarrow 0$ 
4:   for  $i = 1$  to  $m$  do
5:     Compute the loss  $L(f(x_i; \Theta), y_i)$ 
6:      $\hat{g} \leftarrow \hat{g} + \text{gradients of } \frac{1}{m} L(f(x_i; \Theta), y_i) \text{ w.r.t } \Theta$ 
7:    $\Theta \leftarrow \Theta - \eta_t \hat{g}$ 
8: return  $\Theta$ 
```

- Des valeurs plus élevées de m fournissent de meilleures estimations des gradients globaux du corpus, tandis que des valeurs plus petites permettent davantage de mises à jour et donc une convergence plus rapide.
- Pour des tailles modérées de m , certaines architectures de calcul (par exemple les GPU) permettent une implémentation parallèle efficace des calculs effectués aux lignes 3 à 6.

⁰Source:[Goldberg, 2017]

- Hinge (ou perte SVM) : pour les problèmes de classification binaire, la sortie du classificateur est un scalaire $\tilde{y}$ et la sortie attendue y appartient à $\{+1, -1\}$. La règle de classification est $\hat{y} = \text{sign}(\tilde{y})$, et une classification est considérée correcte si $y \cdot \tilde{y} > 0$.

$$L_{\text{hinge(binaire)}}(\tilde{y}, y) = \max(0, 1 - y \cdot \tilde{y})$$

- Entropie croisée binaire (ou perte logistique) : utilisée pour la classification binaire avec des sorties interprétables comme probabilités conditionnelles. La sortie du classificateur $\tilde{y}$ est transformée avec la fonction sigmoïde pour obtenir une valeur dans $[0, 1]$ et interprétée comme la probabilité conditionnelle $P(y = 1|x)$.

$$L_{\text{logistic}}(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

Fonctions de perte

- La perte logistique possède une interprétation probabiliste :
- On suppose que $P(y = 1|\vec{x}; \Theta) = \sigma(f(\vec{x})) = \frac{1}{1+e^{-\vec{x} \cdot \vec{w} + b}}$ et $P(y = 0|\vec{x}; \Theta) = 1 - \sigma(f(\vec{x}))$
- On peut écrire cela de manière plus compacte :

$$P(y|\vec{x}; \Theta) = \sigma(f(\vec{x}))^y \times (1 - \sigma(f(\vec{x})))^{1-y}$$

- L'expression ci-dessus correspond à la fonction de masse de probabilité de la distribution de Bernoulli.
- On remplace maintenant $\sigma(f(\vec{x}))$ par $\hat{y}$:

$$P(y|\vec{x}; \Theta) = \hat{y}^y \times (1 - \hat{y})^{1-y}$$

- Si l'on applique une estimation du maximum de vraisemblance (MLE) à cette expression, on prend le logarithme et on maximise par rapport aux paramètres Θ :

$$y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

- Maximiser cette expression revient à minimiser la perte logistique !
- Beaucoup de fonctions de perte correspondent en fait à la log-vraisemblance négative de modèles probabilistes.

- Perte d'entropie croisée catégorique : utilisée lorsque l'on souhaite une interprétation probabiliste des scores multi-classes. Elle mesure la dissimilarité entre la distribution réelle des étiquettes $\vec{y}$ et la distribution prédite $\vec{\hat{y}}$:

$$L_{\text{cross-entropy}}(\vec{\hat{y}}, \vec{y}) = - \sum_i \vec{y}_{[i]} \log(\vec{\hat{y}}_{[i]})$$

- Lorsqu'on utilise la perte d'entropie croisée, on suppose que la sortie du classificateur est transformée via la fonction softmax.
- La fonction softmax comprime la sortie k -dimensionnelle en valeurs comprises entre 0 et 1, dont la somme vaut 1. Ainsi, $\vec{\hat{y}}_{[i]} = P(y = i|x)$ représente la distribution conditionnelle de l'appartenance à chaque classe.

- Pour les problèmes de classification stricte (hard-classification) où chaque exemple d'entraînement a une seule classe correcte, $\vec{y}$ est un vecteur one-hot représentant la classe vraie.
- Dans ce cas, l'entropie croisée peut être simplifiée en :

$$L_{\text{cross-entropy (classification stricte)}}(\vec{\hat{y}}, \vec{y}) = -\log(\hat{y}_{[t]})$$

où t est la classe correcte de l'exemple.

- Notre problème d'optimisation peut admettre plusieurs solutions et, surtout dans des dimensions élevées, il peut aussi sur-apprendre (overfitting).
- Scénario : dans notre problème d'identification de langue, un des documents du jeu d'entraînement ($\vec{x}_O$) est un outlier.
- Le document est en réalité en allemand, mais il est étiqueté comme français.
- Pour réduire la perte, l'algorithme peut identifier des caractéristiques (bigrammes de lettres) présentes dans très peu d'autres documents.
- L'algorithme donnera alors à ces caractéristiques des poids très forts vers la classe française (incorrecte).

- C'est une mauvaise solution pour le problème d'apprentissage.
- Le modèle apprend quelque chose d'incorrect.
- Des documents allemands de test qui partagent de nombreux mots avec $\vec{x}_O$ pourraient être classés à tort comme français.
- Nous souhaitons contrôler ce type de situation en éloignant l'algorithme de ces solutions erronées.
- Idée : il est acceptable de mal classer quelques exemples s'ils ne s'accordent pas bien avec le reste des données.

- Régularisation : on ajoute un terme de régularisation R à l'objectif d'optimisation.
- L'objectif de ce terme : contrôler la complexité (poids trop grands) des paramètres (Θ) et éviter le sur-apprentissage :

$$\begin{aligned}\hat{\Theta} &= \operatorname{argmin}_{\Theta} \mathcal{L}(\Theta) + \lambda R(\Theta) \\ &= \operatorname{argmin}_{\Theta} \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i) + \lambda R(\Theta)\end{aligned}\tag{14}$$

- Le terme de régularisation considère les valeurs des paramètres et évalue leur complexité.
- La valeur de l'hyperparamètre λ doit être choisie manuellement, en fonction de la performance de classification sur un jeu de développement.

- En pratique, les régularisateurs R associent la complexité à des poids élevés.
- Leur objectif est de maintenir les valeurs des paramètres (Θ) faibles.
- Ils orientent l'apprentissage vers des solutions avec des normes faibles pour les matrices de paramètres (W).
- Les choix les plus courants pour R sont la norme L_2 , la norme L_1 , et l'elastic-net.

- Dans la régularisation L_2 , R prend la forme de la norme L_2 au carré des paramètres.
- Objectif : maintenir la somme des carrés des valeurs des paramètres faible.

$$R_{L_2}(W) = ||W||_2^2 = \sum_{i,j} (W_{[i,j]})^2$$

- Le régularisateur L_2 est aussi appelé prior gaussien ou décroissance des poids (weight decay).
- Les modèles régularisés L_2 sont fortement pénalisés pour des poids de paramètres élevés.
- Une fois les valeurs proches de zéro, leur effet devient négligeable.
- Le modèle préférera diminuer la valeur d'un paramètre avec un poids élevé de 1 que de diminuer la valeur de dix paramètres ayant déjà des poids faibles de 0,1 chacun.

- Dans la régularisation L_1 , R prend la forme de la norme L_1 des paramètres.
- Objectif : maintenir la somme des valeurs absolues des paramètres faible.

$$R_{L_1}(W) = ||W||_1 = \sum_{i,j} |W_{[i,j]}|$$

- Contrairement à L_2 , le régularisateur L_1 pénalise uniformément les valeurs faibles et élevées.
- Il incite à réduire toutes les valeurs non nulles vers zéro.
- Il favorise donc des solutions clairsemées (sparse) — des modèles avec de nombreux paramètres nuls.
- Le régularisateur L_1 est aussi appelé prior sparse ou lasso **tibshirani1996regression**.

- La régularisation elastic-net [Zou and Hastie, 2005] combine à la fois L_1 et L_2 :

$$R_{\text{elastic-net}}(W) = \lambda_1 R_{L_1}(W) + \lambda_2 R_{L_2}(W)$$

- Bien que l'algorithme SGD puisse et produise souvent de bons résultats, des algorithmes plus avancés existent également.
- Les algorithmes SGD+Momentum [Polyak, 1964] et Nesterov Momentum [Nesterov, 2018, Sutskever et al., 2013] sont des variantes de SGD dans lesquelles les gradients précédents sont accumulés et influencent la mise à jour actuelle.
- Les algorithmes à taux d'apprentissage adaptatif tels qu'AdaGrad [Duchi et al., 2011], AdaDelta [Zeiler, 2012], RMSProp [Tieleman and Hinton, 2012], et Adam [Kingma and Ba, 2014] sont conçus pour sélectionner le taux d'apprentissage pour chaque minibatch.
- Cela peut être fait pour chaque coordonnée, réduisant ainsi le besoin d'ajuster manuellement le planning du taux d'apprentissage.
- Pour plus de détails sur ces algorithmes, voir les articles originaux ou [Goodfellow et al., 2016] (Sections 8.3, 8.4).

Jeux d'entraînement, de test et de validation

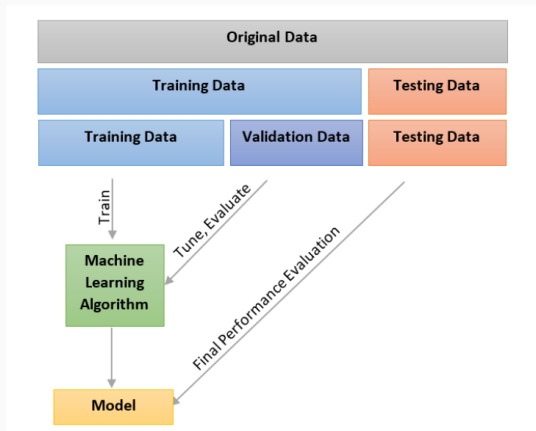
- Lors de l'entraînement d'un modèle, notre objectif est de produire une fonction $f(\vec{x})$ qui associe correctement les entrées $\vec{x}$ aux sorties $\hat{y}$, comme l'indique le jeu d'entraînement.
- La performance sur les données d'entraînement peut être trompeuse : notre objectif est de former une fonction capable de généraliser à des exemples jamais vus.
- Jeu tenu à l'écart (held-out set) : on divise le jeu d'entraînement en sous-ensembles d'entraînement et de test (par exemple 80% et 20%). On entraîne sur l'entraînement et on mesure la précision sur le test.
- Problème : en pratique, on entraîne souvent plusieurs modèles, on compare leur qualité et on choisit le meilleur.
- Choisir le meilleur modèle selon la précision du jeu tenu à l'écart peut donner une estimation trop optimiste de la qualité du modèle.
- On ne sait pas si les paramètres choisis pour le classificateur final sont bons en général, ou seulement pour les exemples du jeu tenu à l'écart.

Jeux d'entraînement, de test et de validation

- La méthodologie acceptée consiste à diviser les données en trois ensembles : entraînement, validation (appelé aussi ensemble de développement) et test¹.
- Cela vous donne deux ensembles tenus à l'écart : un ensemble de validation (ou de développement) et un ensemble de test.
- Toutes les expériences, ajustements, analyses d'erreurs et choix de modèle doivent être basés sur l'ensemble de validation.
- Ensuite, un seul passage du modèle final sur l'ensemble de test donne une bonne estimation de sa qualité attendue sur des exemples jamais vus.
- Il est important de garder l'ensemble de test le plus propre possible, en effectuant le moins d'expérimentations dessus.
- Certains préconisent même de ne jamais regarder les exemples du jeu de test pour ne pas biaiser la conception du modèle.

¹Une approche alternative est la validation croisée, mais elle ne s'adapte pas bien à l'entraînement de réseaux neuronaux profonds.

Jeux d'entraînement, de test et de validation



¹source: <https://www.codeproject.com/KB/AI/1146582/validation.PNG>

Une limitation des modèles linéaires : le problème XOR

- La classe d'hypothèses des modèles linéaires (et log-linéaires) est fortement limitée.
- Par exemple, elle ne peut pas représenter la fonction XOR, définie comme suit :

$$\begin{array}{ll} \text{xor}(0, 0) & = 0 \\ \text{xor}(1, 0) & = 1 \\ \text{xor}(0, 1) & = 1 \\ \text{xor}(1, 1) & = 0 \end{array} \quad (15)$$

Une limitation des modèles linéaires : le problème XOR

- Il n'existe aucun paramétrage $\vec{w} \in \mathcal{R}^2, b \in \mathcal{R}$ tel que :

$$(0, 0) \cdot \vec{w} + b < 0$$

$$(0, 1) \cdot \vec{w} + b \geq 0$$

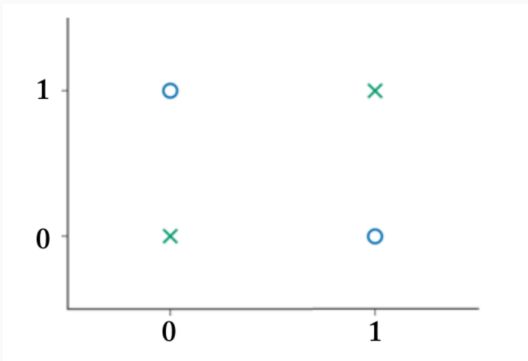
$$(1, 0) \cdot \vec{w} + b \geq 0$$

$$(1, 1) \cdot \vec{w} + b < 0$$

(16)

Une limitation des modèles linéaires : le problème XOR

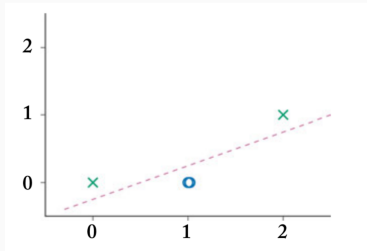
- Pour comprendre pourquoi, considérons le graphique de la fonction XOR, où les cercles bleus représentent la classe positive et les croix vertes la classe négative.



- Il est évident qu'aucune droite ne peut séparer correctement les deux classes.

Transformations non linéaires des entrées

- Si nous transformons les points en les passant par la fonction non linéaire $\phi(x_1, x_2) = [x_1 \times x_2, x_1 + x_2]$, le problème XOR devient linéairement séparable.



- La fonction ϕ a mappé les données dans une représentation adaptée à une classification linéaire.

Transformations non linéaires des entrées

- Nous pouvons maintenant facilement entraîner un classificateur linéaire pour résoudre le problème XOR.

$$\hat{y} = f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b \quad (17)$$

- Problème : nous devons définir manuellement la fonction ϕ .
- Ce processus dépend du jeu de données particulier et demande beaucoup d'intuition humaine.
- Solution : définir une fonction de transformation non linéaire entraînable, et l'entraîner conjointement avec le classificateur linéaire.
- Trouver la représentation adaptée devient alors la responsabilité de l'algorithme d'apprentissage.

Fonctions de transformation entraînaibles

- La fonction de transformation peut prendre la forme d'un modèle linéaire paramétré.
- Suivi d'une fonction d'activation non linéaire g appliquée à chacune des dimensions de sortie :

$$\begin{aligned}\hat{y} &= f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b \\ \phi(\vec{x}) &= g(\vec{x}W' + \vec{b}')\end{aligned}\tag{18}$$

- En prenant $g(x) = \max(0, x)$ et $W' = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $\vec{b}' = (-1 \quad 0)$.
- On obtient une transformation équivalente à $[x_1 \times x_2, x_1 + x_2]$ pour nos points d'intérêt $(0,0)$, $(0,1)$, $(1,0)$, et $(1,1)$.
- Cela résout avec succès le problème XOR !
- L'apprentissage simultané de la fonction de représentation et du classifieur linéaire au-dessus est l'idée principale derrière le deep learning et les réseaux de neurones.
- En fait, l'équation précédente décrit une architecture de réseau de neurones très courante appelée perceptron multi-couches (MLP).

Références

- Abu-Mostafa, Y. S., Magdon-Ismaïl, M., et Lin, H.-T. (2012). *Learning From Data: A Short Course*. AMLBook.
- Duchi, J., Hazan, E., et Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159.
- Goldberg, Y. (2017). *Neural network methods for natural language processing*. Synthesis Lectures on Human Language Technologies, 10(1):1–309.
- Goodfellow, I., Bengio, Y., et Courville, A. (2016). *Deep Learning*. MIT Press.
- Kingma, D. P., et Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- Nesterov, Y. (2018). *Lectures on convex optimization, volume 137*. Springer.
- Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17.
- Sutskever, I., Martens, J., Dahl, G., et Hinton, G. (2013). On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning*, pages 1139–1147.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288.
- Tieleman, T., et Hinton, G. (2012). Lecture 6.5-RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural networks for machine learning, 4(2):26–31.
- Zeiler, M. D. (2012). Adadelata: an adaptive learning rate method. arXiv preprint arXiv:1212.5701.
- Zou, H., et Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):301–320.